UNIT1

PARALLEL DATABASES

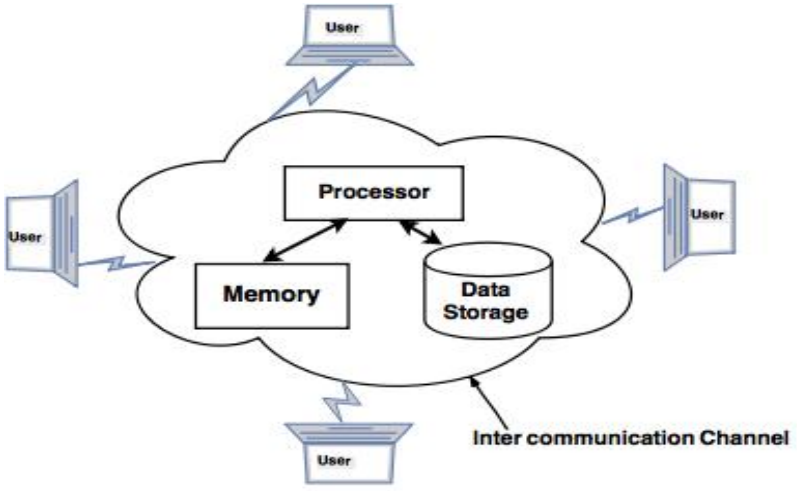
In the ever-expanding world of data, companies face the challenge of managing vast amounts of information at a rapid pace. Traditional approaches like client-server and centralized systems often fall short in terms of efficiency. To address this, the concept of Parallel Databases emerged.

A Parallel Database system enhances the processing of data by utilizing multiple resources simultaneously, such as multiple CPUs and disks operating in parallel. This enables the system to perform various parallelization operations, including data loading and query processing.

Goals of Parallel Databases

The development of Parallel Databases was driven by several key goals:

1. **Improve Performance:** Enhancing system performance is achieved by connecting multiple CPUs and disks in parallel. This approach allows for the simultaneous use of many smaller processors.
2. **Improve Availability of Data:** To enhance data availability, information can be duplicated and stored in multiple locations. For instance, if a module contains a relation (table in a database) that becomes unavailable, it is crucial to make it accessible from another module.
3. **Improve Reliability:** The reliability of the system is bolstered by ensuring the completeness, accuracy, and availability of data.
4. **Provide Distributed Access to Data:** Parallel Database systems facilitate companies with multiple branches in various cities to access data seamlessly. This distributed access ensures efficient utilization of the parallel database architecture.



Parallel database system

Parallel Database Architectures

In today's modern era, everyone is eager to store the information they collect. It's not just big companies; even smaller ones gather data and keep extensive databases. Despite the space these databases take up, they offer significant benefits in various ways. A notable advantage is their role in decision-making through a decision support system.

However, dealing with massive amounts of data using traditional centralized systems can be challenging. Even simple queries can take a lot of time. The solution lies in using Parallel Database Systems, where a table or database is spread across multiple processors, allowing queries to be executed in parallel. This collaborative approach improves system performance, tackling the challenges posed by extensive data.

To effectively implement this solution, specific architectures are necessary. These architectures manage data distribution and enable parallel query execution, leading to improved query and transaction throughput. Figures 1, 2, and 3 depict different architectures proposed and successfully implemented in the realm of Parallel Database Systems. In these figures, P represents Processors, M represents Memory, and D represents Disks/Disk setups.

1. Shared Memory Architecture

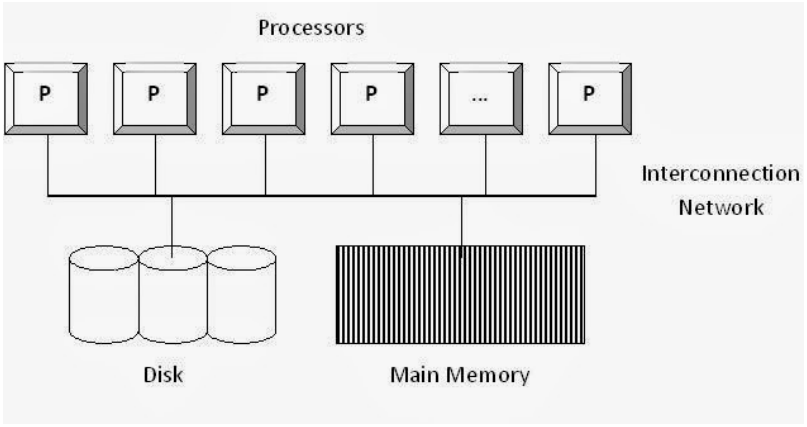


Figure 1 - Shared Memory Architecture

In Shared Memory architecture, multiple processors share a single memory, as illustrated in Figure 1. The figure depicts several processors interconnected through a network with Main memory and disk setup. In parallel systems, the interconnection network is vital, serving as a

high-speed network (such as Bus, Mesh, or Hypercube) that facilitates seamless data sharing among components like Processors, Memory, and Disk.

Advantages:

1. **Simple Implementation:** The setup is uncomplicated, making implementation straightforward.
2. **Effective Communication:** It establishes efficient communication between processors using a single memory address space.
3. **Reduced Communication Overhead:** The single memory address space minimizes communication overhead.

Disadvantages:

1. **Limited Parallelism:** Achieving a higher degree of parallelism (more concurrent operations in different processors) is challenging because all processors share the same interconnection network. This may create a bottleneck, especially in Bus interconnection networks.
2. **Slowdown with Additional Processors:** Adding processors may slow down existing ones due to shared network resources.
3. **Cache-Coherency Maintenance:** Ensuring cache coherency is crucial. If one processor reads data used or modified by others, it must ensure it's the latest version.
4. **Limited Degree of Parallelism:** The degree of parallelism is constrained, and an increase in the number of parallel processes might degrade overall performance.

2. Shared Disk Architecture

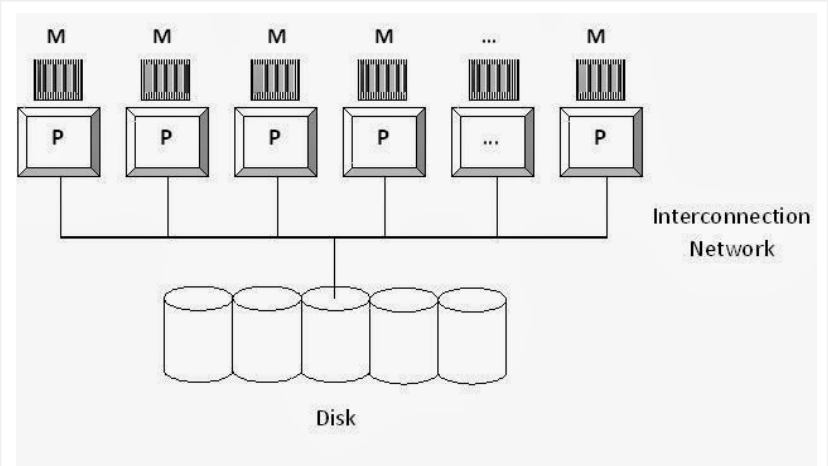


Figure 2 - Shared Disk Architecture

In Shared Disk architecture, all available processors share a single disk or disk setup, while each processor maintains its private memory, as depicted in Figure 2.

Advantages:

1. **Fault Tolerance:** If a processor fails, it doesn't affect the entire system.
2. **Non-Bottleneck Memory Interconnection:** Unlike Shared Memory architecture, the interconnection to memory is not a bottleneck.

3. **Support for More Processors:** This architecture can accommodate a larger number of processors compared to Shared Memory architecture.

Disadvantages:

1. **Disk Interconnection Bottleneck:** Interconnection to the disk becomes a bottleneck because all processors share a common disk setup.
2. **Slow Inter-Processor Communication:** Communication between processors is slow because each processor has its memory. This requires reading data from other processors' memory, needing additional software support.

Example Real-Time Shared Disk Implementation

- **DEC Clusters (VMScluster) running Rdb**

3. Shared Nothing Architecture

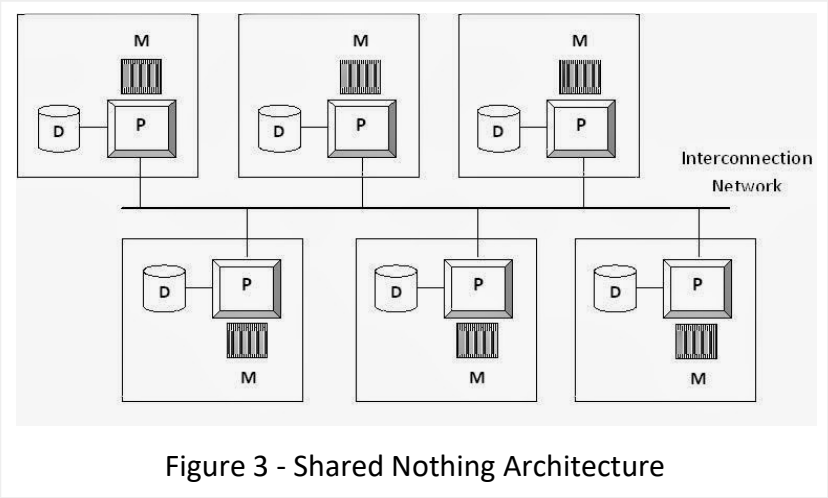


Figure 3 - Shared Nothing Architecture

In Shared Nothing architecture, each processor operates independently with its dedicated memory and disk setup. This setup resembles a group of individual computers connected through a high-speed interconnection network, using standard network protocols and switches for data sharing between the computers. This architecture is commonly used in Distributed Database Systems. In the implementation of Shared Nothing parallel database systems, it's crucial to maintain homogeneity by using similar nodes (although heterogeneous nodes can be employed in a distributed database system).

Advantages:

1. **Scalability:** The number of processors can be scaled, providing flexibility for adding more computers to the system.
2. **Localized Data Requests:** Only data requests that cannot be fulfilled by local processors are forwarded through the interconnection network, unlike other architectures.

Disadvantages:

1. **Costly Non-Local Disk Access:** Accessing non-local disks can be expensive. If a server receives a request and the required data is not available locally, it must be routed to the server where the data is stored, adding complexity to the process.

2. **Communication Costs:** There are communication costs involved in transporting data among computers.

Example Real-Time Shared Nothing Implementations:

- Teradata
- Tandem
- Oracle nCUBE

Data Partitioning Strategies in Parallel Database Systems

Efficiently partitioning tables/databases is a crucial step in parallelizing database activities. By distributing data equally among different processors, better performance and parallelism for the entire system can be achieved. Various data partitioning techniques in parallel databases include:

Horizontal Fragmentation: Partitioning tables based on conditions specified through the WHERE clause of SQL queries to distribute groups of tuples (records). For instance, using the table schema:

STUDENT (Regno, SName, Address, Branch, Phone)

Horizontal partitioning could be done using:

sqlCopy code

```
SELECT * FROM student WHERE Branch = branch name;
```

This doesn't alter the table structure; it merely fragments data based on the specified conditions.

Vertical Fragmentation: Partitioning tables using decomposition rules to break and distribute them into multiple partitions vertically, creating different schemas. For example, breaking the STUDENT table into two different schemas: STUDENT(Regno, SName, Address, Branch) and STU_PHONE(Regno, Phone).

Among these fragmentation techniques, parallel databases primarily involve Horizontal Partitioning, distributing data across processors for simultaneous query execution.

Partitioning Strategies:

Various partitioning strategies proposed to manage data distribution into multiple processors evenly. Assuming n processors P0, P1, P2, …, Pn-1 and n disks D0, D1, D2, …, Dn-1 where data is partitioned. The value of n is chosen according to the degree of parallelism required. The partitioning strategies are,

Round-Robin Partitioning:

- It is the simplest form of partitioning strategy. Here, data is distributed into various disks in a round-robin fashion, ensuring even distribution.

Hash Partitioning:

- This strategy identifies one or more attributes as partitioning attributes. A carefully chosen hash function takes the identified partitioning attributes as input to hash data.

For example, consider the following table:

EMPLOYEE(ENo, EName, DeptNo, Salary, Age)

If we choose DeptNo attribute as the partitioning attribute and have 10 disks to distribute data, the hash function would be:

pythonCopy code

```
h(DeptNo) = DeptNo mod 10
```

Range Partitioning:

- In Range Partitioning, one or more attributes are identified as partitioning attributes, and a range partition vector is chosen to partition the table into n disks.

For example, for the EMPLOYEE relation given above, if the partitioning attribute is Salary, then the vector would be:
pythonCopy code
[5000, 15000, 30000]
These values represent individual salary ranges, creating different disks/partitions.
The above-discussed partition strategies must be chosen carefully according to the workload of your parallel database system. The workload may involve many components like which attribute is frequently used in any queries as a filtering condition, the number of records in a table, the size of the database, the approximate number of incoming queries in a day, etc.

Example Partitioning Strategies:

A. Round-Robin Partitioning:

- In this strategy, records are partitioned in a round-robin manner using the function i mod n, where i is the record position in the table, and n is the number of partitions/disks.

B. Hash Partitioning:

- Taking the GRADE attribute of the Emp_table to explain Hash partitioning. Using a hash function as follows:

pythonCopy code
h(GRADE) = (GRADE mod n)

C. Range Partitioning:

- Considering GRADE of Emp_table to partition under range partitioning. Identifying partitioning vector:

pythonCopy code
[2, 4]
These values represent individual grade ranges, creating different disks/partitions

Interquery and Intraquery Parallelism in Parallel Database

Inter-Query Parallelism involves running multiple Queries or Transactions simultaneously on different processors.

Advantages:

1. **Increased Transaction Throughput:** More transactions can be executed, enhancing overall system efficiency.
2. **Scalability for Transaction Processing:** Particularly suitable for On-Line Transaction Processing (OLTP) systems, scaling up the transaction processing system.

Supported Parallel Database Architectures:

- Easily implemented in Shared Memory Parallel Systems, where lock tables and log information are in the same memory.

- In Shared Disk and Shared Nothing architectures, locking and logging require message passing between processors, which is considered costly. Cache coherency problems may arise.

Example Database Systems supporting Inter-Query Parallelism:

- Oracle 8
- Oracle Rdb

Intra-Query Parallelism

Intra-Query Parallelism focuses on executing a single Query in parallel on multiple processors.

Advantages:

1. **Speeding up Complex Queries:** Especially effective for long-running, complex queries.
2. **Ideal for Scientific Calculations:** Suited for complex scientific calculations within queries.

Supported Parallel Database Architectures:

- Supported in Shared Memory, Shared Disk, and Shared Nothing parallel architectures. Locking and logging concerns are minimized as it involves parallelizing a single query.

Types:

1. **Intra-Operation Parallelism:** Speeds up a query by parallelizing the execution of individual operations such as Sort, Join, Projection, and Selection.
2. **Inter-Operation Parallelism:** Accelerates a query by parallelizing various operations within the query. For instance, a query involving the join of four tables can be executed in parallel on two processors, each handling two relations locally.

Example Database Systems supporting Intra-Query Parallelism:

- Informix
- Teradata

Parallel Database - Intra-Query Parallelism

Intra-Query Parallelism Explained: Executing a single query in parallel using multiple processors or disks involves dividing the data into smaller units and performing the query on these smaller tables. This approach is particularly beneficial for handling complex queries that consume significant time and resources.

For example, consider the query given below:

sqlCopy code

```
SELECT * FROM Email ORDER BY Start_Date;
```

This query will sort the records of the Email table in ascending order based on the Start_Date attribute. If the same query is executed in several processors simultaneously, it can finish sorting in less time. For instance, if 10,000 records are distributed equally into 10 processors, each processor can sort 1,000 records more efficiently, later combining them to produce the final sorted result.

In Intraquery Parallelism, we have two types:

- **Intraoperation Parallelism**
- **Interoperation Parallelism**

Parallel Database - Intraoperation Parallelism

Intra-operation Parallelism

It involves parallelizing a single relational operation given in a query.

For example, in the query:

sqlCopy code

SELECT * FROM Email ORDER BY Start_Date;

The relational operation is Sorting. Since a table may have a large number of records, the operation can be performed on different subsets of the table in multiple processors, reducing the time required to sort.

Consider another query:

sqlCopy code

SELECT * FROM Student, CourseRegd WHERE Student.Regno = CourseRegd.Regno;

In this query, the relational operation is Join. Parallelism is required here if the size of tables is very large. If we exploit parallelism, we could achieve better performance.

There are many such relational operations that can be executed in parallel using many processors on subsets of the table/tables mentioned in the query.

Parallel Sort - Range Partitioning Sort

Range Partitioning Sort

Assumptions: Assume n processors, P0, P1, …, Pn-1, and n disks D0, D1, …, Dn-1. Disk Di is associated with Processor Pi. Relation R is partitioned into R0, R1, …, Rn-1 using Round-robin technique or Hash Partitioning technique or Range Partitioning technique (if range partitioned on some other attribute other than sorting attribute)

Objective: To sort a relation (table) Ri that resides on n disks on an attribute A in parallel.

Steps:

1. Partition the relations Ri on the sorting attribute A at every processor using a range vector v. Send the partitioned records that fall in the ith range to Processor Pi where they are temporarily stored in Di.
2. Sort each partition locally at each processor Pi. And, send the sorted results for merging with all the other sorted results, which is a trivial process.

Parallel Database - Parallel External Sort-Merge Technique

Parallel External Sort-Merge

Assumptions: Assume n processors, P0, P1, …, Pn-1, and n disks D0, D1, …, Dn-1. Disk Di is associated with Processor Pi. Relation R is partitioned into R0, R1, …, Rn-1 using Round-robin technique or Hash Partitioning technique or Range Partitioning technique (partitioned on any attribute)

Objective: To sort a relation (table) Ri on an attribute A in parallel where R resides on n disks.

Steps:

1. Sort the relation partition Ri, which is stored on disk Di on the sorting attribute of the query.

2. Identify a range partition vector v and range partition every Ri into processors, P0, P1, …, Pn-1 using vector v.
3. Each processor Pi performs a merge on the incoming range partitioned data from every other processor (The data are actually transferred in order. That is, all processors send the first partition into P0, then all processors send the second partition into P1, and so on).
4. Finally, concatenate all the sorted data from different processors to get the final result.

Point to note: Range partition must be done using a good range-partitioning vector. Otherwise, skew might be a problem.

Example: Consider the relation schema Employee:

Employee (Emp_ID, EName, Salary)

Assume that the relation Employee is permanently partitioned using the Round-robin technique into 3 disks D0, D1, and D2, which are associated with processors P0, P1, and P2. This initial state is given in Figure 1.

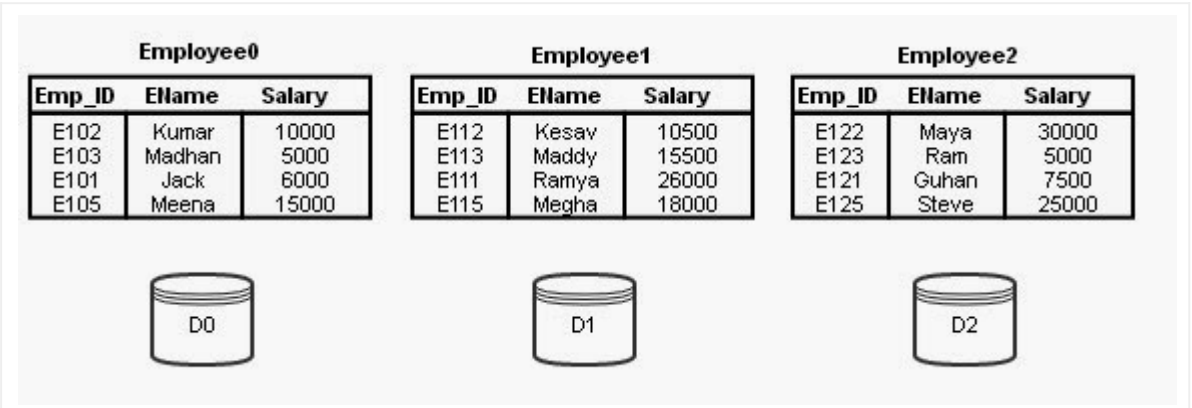


Figure 1 - Partitioned Employee

Parallel Join Technique - Partitioned Join

Partitioned Join involves partitioning relational tables based on joining attributes and executing join operations in parallel.

How Partitioned Join Works:

Assume we have relational tables r and s to be joined using attributes r.A and s.B. The system partitions both r and s into n partitions, using the same partitioning technique. Attributes A and B are used as partitioning attributes for r and s, respectively. The system sends partitions ri and si to processor Pi, where the join is performed locally.

Types of Joins with Partitioned Join:

Only joins such as Equi-Joins and Natural Joins can be performed using Partitioned Join.

Why Equi-Join and Natural Join?

Equi-Join or Natural Join is done between two tables using an equality condition like r.A = s.B. Partitioning relations r and s based on attributes A and B ensures that tuples with the same value for A and B end up in the same partition.

Fragment and Replicate Join - A Parallel Joining Technique

Introduction:

Joining tables through partitioning works only for Equi-Joins or natural joins. For non-equal joins, consider a join condition like r.a > s.b. In such cases, where records from one relation must join with some records of the other relation based on a non-equal condition, Partitioned Join won't work.

Fragment and Replicate Join:

Fragmenting means partitioning a table horizontally or vertically, while replicating means duplicating the relation. This join technique involves fragmenting and replicating tables to be joined.

Asymmetric Fragment and Replicate Join:

1. The system fragments table r into n partitions.
2. The system replicates the other table s into n processors.
3. Processors perform the join locally on ri and s.

Points to Note:

1. Non-equal join can be performed in parallel.
2. Best suited when one relation is already partitioned into n processors.
3. Any partitioning techniques can be used.
4. Efficient if one relation is very small.

Fragment and Replicate Join

General Case:

1. The system fragments table r into m fragments.
2. The system fragments table s into n fragments.
3. Distribute all partitions of r and s into available processors.

Note:

- Best suited when one relation is small and can fit into memory.
- For large, non-equal joins, use Fragment and Replicate Join.

Partitioned Parallel Hash Join

The Sequential Hash Join technique can be parallelized.

The Technique:

Assume:

- n processors, P0, P1, …, Pn-1.
- Two tables, r and s.
- s is the smaller table.

Interoperation Parallelism:

It involves executing different operations of a query in parallel.

Pipelined Parallelism:

In Pipelined Parallelism, results produced by one operation are consumed by the next operation in the pipeline. It works well with a small number of processors but not for high degrees of parallelism.

Independent Parallelism:

Operations that are not dependent on each other can be executed in parallel at different processors. It does not work well with a high degree of parallelism.

Conclusion:

Choose the parallelization technique based on the nature of the operations and the degree of parallelism needed for optimal performance

Query Optimization

Optimizing queries is a key factor in ensuring the efficiency of relational technology. When it comes to queries, a query optimizer plays a crucial role by seeking the most cost-effective execution plan from various possibilities that produce the same result. While optimizing queries for sequential evaluation is a known task, the complexity increases when dealing with parallel evaluation.

Challenges in Parallel Query Evaluation:

1. **Complex Cost Models:**

- Consideration of partitioning costs is essential.
- Factors like skew and resource contention add layers of complexity to cost models.

2. **Parallelizing Queries:**
    - Decision-making on how each operation should be parallelized.
    - Determining the optimal number of processors to use.
3. **Scheduling Execution Tree:**
    - Critical decisions on pipelining across processors, independent execution, or sequential execution.
4. **Resource Allocation:**
    - Allocating resources, such as processors, disks, and memory, to each operation in the tree.

Designing Parallel Systems:

Efficiently loading data in parallel from external sources is vital for handling large volumes of incoming data. In large parallel database systems, addressing availability issues is crucial, including resilience to processor or disk failures and online reorganization of data and schema changes.

Parallelism on Multicore Processors:

With parallelism becoming widespread in today's computers, most database systems operate on a parallel platform. This shift is a response to the trend in computer architecture where processors, even in smaller devices, have multiple cores on one chip.

Moore's Law and Processor Speed:

- Moore's Law observes the exponential increase in processor speed, doubling every 18 to 24 months.
- Modern processors often consist of multiple cores on a chip, known as a multicore processor.

Cache Memory and Multithreading:

- Each core processes an independent stream of machine instructions.

- Cache memory is introduced to overcome the bottleneck of accessing data from main memory.
- Cache levels, labeled L1, L2, etc., form a hierarchy below main memory, with L1 being the fastest but most costly.

Adapting Database System Design:

Although database systems seem ideal for leveraging numerous cores and threads due to their support for concurrent transactions, adapting database system design and query processing to multicore and multithreaded systems remains an active area of research